Name : Jheboy B. Asid

Data Modeling & Database Systems

In this lab, you will apply SQL to query and analyze data stored in a database. By the end of the lab, you will be able to extract data using a variety of SQL commands and store the results for further analysis.

Tips for completing this lab

As you navigate this lab, keep the following tips in mind:

- `-- YOUR CODE HERE` indicates where you should write code. Note: SQL uses the double dash for comments, as opposed to Python's pound sign (#). Be sure to replace this with your own code before running the code cell.
- Use `%sql` to execute *single-line* SQL commands within a Jupyter Notebook cell.
- For *multi-line* SQL commands, such as creating tables or complex queries, use `%%sql` at the beginning of the cell.
- When you are asked "get all customers", you can safely assume you can use `SELECT *` to show all fields. If you are directed to "get the names of the customers", on the other hand, you can assume you need to include the field names in the `SELECT` statement.
- You can save your work manually by clicking File and then Save in the menu bar at the top of the notebook.
- You can download your work locally by clicking File and then Download and then specifying your preferred file format in the menu bar at the top of the notebook.

Scenario

You are a data analyst working for a bookstore. You have access to a database containing information about customers, books, and reviews. Your goal is to use SQL to analyze customer data, identify trends, and extract valuable insights that can be used to improve the bookstore's marketing and sales strategies. Before you can do that, you want to familiarize yourself with the data, so you will run some queries to examine the data format.

Tasks

Initial Setup

Run the following cells to set up this Jupyter Notebook lab for working with SQL.

- Installs `ipython-sql`, a magic extension for Jupyter Notebooks that allows you to write and execute SQL queries directly within notebook cells.
- Loads the `ipython-sql` extension.
- Establishes a connection between your Jupyter Notebook and your SQLite database `mydatabase.db`.
- Configures the display style for SQL query results to ensure compatibility with the formatting expected by this notebook.

In [1]:
```
pip install ipython-sql
```

```
Collecting ipython-sql
  Downloading ipython_sql-0.5.0-py3-none-any.whl.metadata (17 kB)
Collecting prettytable (from ipython-sql)
  Downloading prettytable-3.17.0-py3-none-any.whl.metadata (34 kB)
Requirement already satisfied: ipython in c:\users\jheboy\anaconda3\envs\sd11-cpe2a
\lib\site-packages (from ipython-sql) (9.7.0)
Collecting sqlalchemy>=2.0 (from ipython-sql)
  Downloading sqlalchemy-2.0.46-cp313-cp313-win_amd64.whl.metadata (9.8 kB)
Collecting sqlparse (from ipython-sql)
  Downloading sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Requirement already satisfied: six in c:\users\jheboy\anaconda3\envs\sd11-cpe2a\lib
\site-packages (from ipython-sql) (1.17.0)
Collecting ipython-genutils (from ipython-sql)
  Downloading ipython_genutils-0.2.0-py2.py3-none-any.whl.metadata (755 bytes)
Collecting greenlet>=1 (from sqlalchemy>=2.0->ipython-sql)
  Downloading greenlet-3.3.2-cp313-cp313-win_amd64.whl.metadata (3.8 kB)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\jheboy\anaconda3
\envs\sd11-cpe2a\lib\site-packages (from sqlalchemy>=2.0->ipython-sql) (4.15.0)
Requirement already satisfied: colorama>=0.4.4 in c:\users\jheboy\anaconda3\envs\sd1
1-cpe2a\lib\site-packages (from ipython->ipython-sql) (0.4.6)
Requirement already satisfied: decorator>=4.3.2 in c:\users\jheboy\anaconda3\envs\sd
11-cpe2a\lib\site-packages (from ipython->ipython-sql) (5.2.1)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in c:\users\jheboy\ana
conda3\envs\sd11-cpe2a\lib\site-packages (from ipython->ipython-sql) (1.1.1)
Requirement already satisfied: jedi>=0.18.1 in c:\users\jheboy\anaconda3\envs\sd11-c
pe2a\lib\site-packages (from ipython->ipython-sql) (0.19.2)
Requirement already satisfied: matplotlib-inline>=0.1.5 in c:\users\jheboy\anaconda3
\envs\sd11-cpe2a\lib\site-packages (from ipython->ipython-sql) (0.2.1)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in c:\users\jheboy\anac
onda3\envs\sd11-cpe2a\lib\site-packages (from ipython->ipython-sql) (3.0.52)
Requirement already satisfied: pygments>=2.11.0 in c:\users\jheboy\anaconda3\envs\sd
11-cpe2a\lib\site-packages (from ipython->ipython-sql) (2.19.2)
Requirement already satisfied: stack_data>=0.6.0 in c:\users\jheboy\anaconda3\envs\s
d11-cpe2a\lib\site-packages (from ipython->ipython-sql) (0.6.3)
Requirement already satisfied: traitlets>=5.13.0 in c:\users\jheboy\anaconda3\envs\s
d11-cpe2a\lib\site-packages (from ipython->ipython-sql) (5.14.3)
Requirement already satisfied: wcwidth in c:\users\jheboy\anaconda3\envs\sd11-cpe2a
\lib\site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython->ipython-sql) (0.2.1
4)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in c:\users\jheboy\anaconda3\envs
\sd11-cpe2a\lib\site-packages (from jedi>=0.18.1->ipython->ipython-sql) (0.8.5)
Requirement already satisfied: executing>=1.2.0 in c:\users\jheboy\anaconda3\envs\sd
11-cpe2a\lib\site-packages (from stack_data>=0.6.0->ipython->ipython-sql) (2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in c:\users\jheboy\anaconda3\envs\sd
11-cpe2a\lib\site-packages (from stack_data>=0.6.0->ipython->ipython-sql) (3.0.0)
Requirement already satisfied: pure_eval in c:\users\jheboy\anaconda3\envs\sd11-cpe2
a\lib\site-packages (from stack_data>=0.6.0->ipython->ipython-sql) (0.2.3)
Downloading ipython_sql-0.5.0-py3-none-any.whl (20 kB)
Downloading sqlalchemy-2.0.46-cp313-cp313-win_amd64.whl (2.1 MB)
   ---------------------------------------- 0.0/2.1 MB ? eta -:--:--
   ---- ----------------------------------- 0.3/2.1 MB ? eta -:--:--
   ---------------------------------- ----- 1.8/2.1 MB 7.3 MB/s eta 0:00:01
   ---------------------------------------- 2.1/2.1 MB 6.4 MB/s  0:00:00
Downloading greenlet-3.3.2-cp313-cp313-win_amd64.whl (230 kB)
Downloading ipython_genutils-0.2.0-py2.py3-none-any.whl (26 kB)
Downloading prettytable-3.17.0-py3-none-any.whl (34 kB)
```

```
Downloading sqlparse-0.5.5-py3-none-any.whl (46 kB)
Installing collected packages: ipython-genutils, sqlparse, prettytable, greenlet, sq
lalchemy, ipython-sql

   ------ ---------------------------------- 1/6 [sqlparse]
   ------------- --------------------------- 2/6 [prettytable]
   -------------------- ------------------- 3/6 [greenlet]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   -------------------------- ------------- 4/6 [sqlalchemy]
   ---------------------------------------- 6/6 [ipython-sql]

Successfully installed greenlet-3.3.2 ipython-genutils-0.2.0 ipython-sql-0.5.0 prett
ytable-3.17.0 sqlalchemy-2.0.46 sqlparse-0.5.5
Note: you may need to restart the kernel to use updated packages.
```

In [2]:

```
%load_ext sql
```

In [3]:

```
%sql sqlite:///mydatabase.db
```

In [4]:

```
%config SqlMagic.style = '_DEPRECATED_DEFAULT'
```

1. Load and investigate existing data

The Coursera Bookstore has three tables: Customer, Book, and Review. In the first step, you will write a simple SELECT query to see the contents of each table

Step 1.1: View the Customer table

To see all the fields in the Customer table, you can issue the following command:

In [5]:

```
%%sql
SELECT *
```

```
FROM Customer;
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[5]:

customer_id	name	email	address	city	state	zip	phone
1	John Smith		123 Main St	Springfield	NC	27263	555-123-4567
2	Aisha Khan		456 Elm St	Springfield	IL	60654	555-987-6543
3	Li Wei		789 Oak St	New City	NY	10956	555-456-7890
4	Carlos Rodriguez		1011 Pine St	Sunnyvale	CA	94086	555-321-0987
5	Kimiko Tanaka		1213 Willow St	Midland	TX	79701	555-789-0123
6	Kwame Asante		1415 Maple St	Baton Rouge	LA	70802	555-234-5678
7	Elena Petrova		1617 Birch St	Anchorage	AK	99501	555-678-9012
8	Pierre Dubois		1819 Cedar St	Boise	ID	83702	555-098-7654
9	Ananya Patel		2021 Oak St	Charlotte	NC	28202	555-567-8901
10	Alessandro Rossi		2223 Pine St	Des Moines	IA	50309	555-890-1234
11	Fatima Ali		2425 Willow St	Honolulu	HI	96813	555-345-6789
12	Dimitris Papadopoulos		2627 Maple St	Jackson	MS	39201	555-765-4321
13	Ayumi Sato		2829 Birch St	Lincoln	NE	68508	555-123-4567
14	Ola Nordmann		3031 Cedar St	Madison	WI	53703	555-987-6543

customer_id	name	email	address	city	state	zip	phone
15	Pablo Gonzalez		3233 Oak St	Nashville	TN	37203	555-456-7890
16	Nkechi Okonkwo		3435 Pine St	Olympia	WA	98501	555-321-0987
17	Seamus Reilly		3637 Willow St	Providence	RI	02903	555-789-0123
18	Astrid Johansen		3839 Maple St	Richmond	VA	23219	555-234-5678
19	Nguyen Van		4041 Birch St	Salt Lake City	UT	84101	555-678-9012
20	Zahara Ben-Ami		4243 Cedar St	Tallahassee	FL	32301	555-098-7654
21	Maria Garcia		4445 Elm St	Hartford	CT	06106	555-543-2109
22	Tao Lin		4647 Oak St	Helena	MT	59601	555-876-5432
23	Abdul Rahman		4849 Pine St	Indianapolis	IN	46204	555-210-9876
24	Sofia Papadopoulou		5051 Willow St	Juneau	AK	99801	555-654-3210
25	Ingrid Svensson		5253 Maple St	Kansas City	MO	64106	555-098-7654
26	Jamal Malik		5455 Birch St	Lansing	MI	48911	555-567-8901
27	Francesca Ferrari		5657 Cedar St	Montgomery	AL	36104	555-890-1234
28	Kofi Kingston		5859 Oak St	Montpelier	VT	05602	555-345-6789

customer_id	name	email	address	city	state	zip	phone
29	Isabella Costa		6061 Pine St	Oklahoma City	OK	73102	555-765-4321
30	Sarah Jones		876 Elm St	Beverly Hills	CA	90210	555-999-8888

If you have issued the command properly, you will see information about the customer, including their name, email, address, and phone number. There should be 30 customers in the database.

Step 1.2: View the Book table

To see all the fields in the Book table, you can use a similar command. Give it a try. Note that SQL uses the double dash (--) to represent a comment, so where you see `-- YOUR CODE HERE`, that indicates you should add your code.

In [6]:
```
%%sql
SELECT *
FROM Book;
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[6]:

book_id	title	author
1	The Great Gatsby	F. Scott Fitzgerald
2	To Kill a Mockingbird	Harper Lee
3	Pride and Prejudice	Jane Austen
4	1984	George Orwell
5	The Catcher in the Rye	J.D. Salinger
6	The Lord of the Rings	J.R.R. Tolkien
7	Harry Potter and the Sorcerer's Stone	J.K. Rowling
8	The Hobbit	J.R.R. Tolkien
9	And Then There Were None	Agatha Christie
10	The Da Vinci Code	Dan Brown
11	The Little Prince	Antoine de Saint-Exupéry
12	The Alchemist	Paulo Coelho
13	The Hitchhiker's Guide to the Galaxy	Douglas Adams
14	The Chronicles of Narnia	C.S. Lewis
15	Animal Farm	George Orwell
16	Jane Eyre	Charlotte Brontë
17	Moby Dick	Herman Melville
18	War and Peace	Leo Tolstoy
19	Hamlet	William Shakespeare
20	The Odyssey	Homer

In [7]:
```
# Checking Your Results
```

There are 20 books in the table.

Step 1.3: View Book names

Using `SELECT *` will display all fields. Instead of displaying all fields in the Book table, assume you want to display only the book titles. Notice in the previous step, the book title field was named title. Modify your query to display only the titles from the Book table.

In [8]:
```
%%sql
SELECT title
FROM Book;
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[8]:

title
The Great Gatsby
To Kill a Mockingbird
Pride and Prejudice
1984
The Catcher in the Rye
The Lord of the Rings
Harry Potter and the Sorcerer's Stone
The Hobbit
And Then There Were None
The Da Vinci Code
The Little Prince
The Alchemist
The Hitchhiker's Guide to the Galaxy
The Chronicles of Narnia
Animal Farm
Jane Eyre
Moby Dick
War and Peace
Hamlet
The Odyssey

In [9]:
```
# Checking Your Results
```

Step 1.4: View Review table

There is one more table to examine. This table, the Review table, will be more difficult to interpret. Write a query similar to Step 1.1 to display all fields in the Review table. This will be examined in a later step.

In [10]:
```
%%sql
SELECT * FROM Review;
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[10]:

review_id	rating	comment	book_id	customer_id
1	5	A timeless classic!	1	3
2	3	Interesting, but not my favorite.	3	17
3	4	Thought-provoking and impactful.	4	22
4	2	Did not connect with the characters.	5	7
5	5	An epic journey!	6	12
6	4	Magical and captivating.	7	25
7	3	A fun adventure.	8	6
8	5	Kept me guessing until the end!	9	15
9	2	Too predictable.	10	8
10	4	Beautiful and heartwarming.	11	19
11	5	Inspirational and thought-provoking.	12	2
12	4	Hilarious and absurd.	13	30
13	3	A childhood favorite.	14	11
14	5	A powerful allegory.	15	21
15	4	A classic for a reason.	16	5
16	2	Did not enjoy the writing style.	17	14
17	5	A masterpiece of literature.	18	28
18	3	Tragic and moving.	19	9
19	4	An epic tale.	20	16
20	5	A must-read for Python enthusiasts.	1	23
21	4	Great for learning the basics.	1	12
22	3	Could be more concise.	1	6
23	5	Excellent reference material.	1	20
24	4	A thought-provoking read.	3	15
25	3	Not as good as the first book.	6	24
26	5	The best in the series.	7	18
27	2	Disappointing ending.	9	5
28	4	A page-turner.	10	27
29	5	A classic whodunit.	9	13
30	3	Slow start, but picks up.	12	26

review_id	rating	comment	book_id	customer_id
31	4	Beautifully written.	11	3
32	5	Changed my perspective.	12	10
33	2	Confusing plot.	13	1
34	4	Loved the characters.	14	29
35	3	Important message.	15	22
36	5	A timeless story.	16	17
37	2	Too long.	17	4
38	4	Gripping and emotional.	18	20
39	3	Left me wanting more.	19	8
40	5	A classic adventure.	20	11

In [11]:
```
# Checking Your Results
```

2. Analyze Customer table using single-table queries

The first step with familiarizing yourself with SQL is to learn how to pull information from a single table. You will practice using query criteria using the WHERE clause in this step.

Step 2.1: Display all customers in the state of New York (NY)

You need to get a list of all customers in the state of New York. Run the following query and notice there are no results:

In [12]:
```
%%sql
SELECT *
FROM Customer
WHERE state = 'New York';
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[12]:

customer_id	name	email	address	city	state	zip	phone

An important part of database operations is ensuring you know what your field values look like. In this database, the customer's state is stored by the abbreviation (NY) rather than name (New York). Revise the query to correctly display customers in NY. There should be one customer in NY.

In [13]:
```
%%sql
SELECT *
FROM Customer
WHERE state = 'NY';
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[13]:

customer_id	name	email	address	city	state	zip	phone
3	Li Wei		789 Oak St	New City	NY	10956	555-456-7890

In [14]:
```
# Checking Your Results
```

Step 2.2: Display all customers in the Springfield, NC

In the case of this small dataset, there are not many customers. Let's say you want to get a list of customers in the town of Springfield, North Carolina. Running the following query will result in two results, even though one customer is not in North Carolina:

In [15]:
```
%%sql
SELECT *
FROM Customer
WHERE city = 'Springfield';
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[15]:

customer_id	name	email	address	city	state	zip	phone
1	John Smith		123 Main St	Springfield	NC	27263	555-123-4567
2	Aisha Khan		456 Elm St	Springfield	IL	60654	555-987-6543

Modify the query so you only see the customers in Springfield, NC but not Springfield, IL. You can use an AND condition to do this.

In [16]:
```
%%sql
SELECT *
FROM Customer
WHERE city = 'Springfield' AND state = 'NC'
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[16]:

customer_id	name	email	address	city	state	zip	phone
1	John Smith		123 Main St	Springfield	NC	27263	555-123-4567

In [17]:
```
# Checking Your Results
```

This is a scenario where you may need to combine two conditions.

Step 2.3: Display all customers in certain states

Imagine you have a new sales rep covering the states Hawaii (HI) and Alaska (AK). You may need to provide them a list of customers in their states. Run the following query and note there are no results:

In [18]:
```
%%sql
SELECT *
FROM Customer
WHERE state = 'HI' AND state = 'AK';
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[18]:

customer_id	name	email	address	city	state	zip	phone

This is a very common misinterpretation in queries. You want a list of customers in Hawaii and Alaska, but each individual customer will only be in either Hawaii or Alaska. Replace the AND with OR, and notice the result set is now 3 customers:

In [19]:
```
%%sql
SELECT *
FROM Customer
WHERE state = 'HI' OR state = 'AK';
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[19]:

customer_id	name	email	address	city	state	zip	phone
7	Elena Petrova		1617 Birch St	Anchorage	AK	99501	555-678-9012
11	Fatima Ali		2425 Willow St	Honolulu	HI	96813	555-345-6789
24	Sofia Papadopoulou		5051 Willow St	Juneau	AK	99801	555-654-3210

In [20]:
```
# Checking Your Results
```

This works well if there are two states. However, imagine this representative is assigned all customers in Hawaii (HI), Alaska (AK), Idaho (ID), Washington (WA), and California (CA). In this situation, you can use the IN operator to specify a list of potential acceptable values.

In [21]:
```
%%sql
SELECT *
FROM Customer
WHERE state IN ('HI','AK','ID','WA','CA');
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[21]:

customer_id	name	email	address	city	state	zip	phone
4	Carlos Rodriguez		1011 Pine St	Sunnyvale	CA	94086	555-321-0987
7	Elena Petrova		1617 Birch St	Anchorage	AK	99501	555-678-9012
8	Pierre Dubois		1819 Cedar St	Boise	ID	83702	555-098-7654
11	Fatima Ali		2425 Willow St	Honolulu	HI	96813	555-345-6789
16	Nkechi Okonkwo		3435 Pine St	Olympia	WA	98501	555-321-0987
24	Sofia Papadopoulou		5051 Willow St	Juneau	AK	99801	555-654-3210
30	Sarah Jones		876 Elm St	Beverly Hills	CA	90210	555-999-8888

In [22]:
```
# Checking Your Results
```

Step 2.4: Match patterns

There are times where you may not want to match something exactly, and this is where the LIKE command is appropriate. Recall you have two wildcard operators, %, and _, that can be used. Let's say you want to find a list of customers who live in a city starting with the letter 'S'. You can use the LIKE command with a wildcard to get this data. Depending on your database management system, capitalization may or may not matter, so make sure you use a capital S for your search.

In [23]:
```
%%sql
SELECT *
FROM Customer
WHERE city LIKE 'S%';
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[23]:

customer_id	name	email	address	city	state	zip	phone
1	John Smith		123 Main St	Springfield	NC	27263	555-123-4567
2	Aisha Khan		456 Elm St	Springfield	IL	60654	555-987-6543
4	Carlos Rodriguez		1011 Pine St	Sunnyvale	CA	94086	555-321-0987
19	Nguyen Van		4041 Birch St	Salt Lake City	UT	84101	555-678-9012

In [24]:
```
# Checking Your Results
```

You can also use two wildcards in the same LIKE. For example, let's say you want to find customers who have the number '9' anywhere in their phone number. You can use two wildcard operators to do this. Try to generate this list:

In [25]:
```
%%sql
SELECT *
FROM Customer
WHERE phone LIKE '%9%';
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[25]:

customer_id	name	email	address	city	state	zip	phone
2	Aisha Khan		456 Elm St	Springfield	IL	60654	555-987-6543
3	Li Wei		789 Oak St	New City	NY	10956	555-456-7890
4	Carlos Rodriguez		1011 Pine St	Sunnyvale	CA	94086	555-321-0987
5	Kimiko Tanaka		1213 Willow St	Midland	TX	79701	555-789-0123
7	Elena Petrova		1617 Birch St	Anchorage	AK	99501	555-678-9012
8	Pierre Dubois		1819 Cedar St	Boise	ID	83702	555-098-7654
9	Ananya Patel		2021 Oak St	Charlotte	NC	28202	555-567-8901
10	Alessandro Rossi		2223 Pine St	Des Moines	IA	50309	555-890-1234
11	Fatima Ali		2425 Willow St	Honolulu	HI	96813	555-345-6789
14	Ola Nordmann		3031 Cedar St	Madison	WI	53703	555-987-6543
15	Pablo Gonzalez		3233 Oak St	Nashville	TN	37203	555-456-7890
16	Nkechi Okonkwo		3435 Pine St	Olympia	WA	98501	555-321-0987
17	Seamus Reilly		3637 Willow St	Providence	RI	02903	555-789-0123
19	Nguyen Van		4041 Birch St	Salt Lake City	UT	84101	555-678-9012

customer_id	name	email	address	city	state	zip	phone
20	Zahara Ben-Ami		4243 Cedar St	Tallahassee	FL	32301	555-098-7654
21	Maria Garcia		4445 Elm St	Hartford	CT	06106	555-543-2109
23	Abdul Rahman		4849 Pine St	Indianapolis	IN	46204	555-210-9876
25	Ingrid Svensson		5253 Maple St	Kansas City	MO	64106	555-098-7654
26	Jamal Malik		5455 Birch St	Lansing	MI	48911	555-567-8901
27	Francesca Ferrari		5657 Cedar St	Montgomery	AL	36104	555-890-1234
28	Kofi Kingston		5859 Oak St	Montpelier	VT	05602	555-345-6789
30	Sarah Jones		876 Elm St	Beverly Hills	CA	90210	555-999-8888

In [26]:
```
# Checking Your Results
```

Finally, though it is a strange example here, you can use both wildcard operators together. Let's say you need to find all customers who have the letter 'a' as the second letter in their name (so, Carlos Rodriguez should appear, but not Elena Petrova). You can use both wildcard operators together to accomplish this:

In [27]:
```
%%sql
SELECT *
FROM Customer
WHERE name LIKE '_a%';
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[27]:

customer_id	name	email	address	city	state	zip	phone
4	Carlos Rodriguez		1011 Pine St	Sunnyvale	CA	94086	555-321-0987
11	Fatima Ali		2425 Willow St	Honolulu	HI	96813	555-345-6789
15	Pablo Gonzalez		3233 Oak St	Nashville	TN	37203	555-456-7890
20	Zahara Ben-Ami		4243 Cedar St	Tallahassee	FL	32301	555-098-7654
21	Maria Garcia		4445 Elm St	Hartford	CT	06106	555-543-2109
22	Tao Lin		4647 Oak St	Helena	MT	59601	555-876-5432
26	Jamal Malik		5455 Birch St	Lansing	MI	48911	555-567-8901
30	Sarah Jones		876 Elm St	Beverly Hills	CA	90210	555-999-8888

In [28]:
```
# Checking Your Results
```

3. Grouping data

As you have seen as part of your studies, statistics are a critical piece of data science. For example, you may want to get statistics such as COUNT, SUM, MIN, MAX, and AVG. You may also want to group this data (sum of all states by state).

Step 3.1: Get count of customers

Especially when it comes to web development, you will need a way to find out how many customers exist in a table. This can be useful to see how many results the server should expect. Write a query to return the count of customers in the Customer table:

In [29]:
```
%%sql
SELECT customer_id, COUNT(*)
FROM Customer;
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[29]:

customer_id	COUNT(*)
1	30

In [30]:
```
# Checking Your Results
```

Step 3.2: Get count of customers in a specific state

Alter your previous query to display the total number of customers in the state of Nebraska (NE). There should be 1 customer.

In [31]:
```
%%sql
SELECT state, COUNT(*)
FROM Customer
where state='NE';
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[31]:

state	COUNT(*)
NE	1

In [32]:
```
# Checking Your Results
```

Step 3.3: Get aggregate totals for all states

There may be times in data science where you need to get results grouped by a certain field. Let's say you need to get a count of customers in each state. This will require adding grouping to your query. At this point, you should also display the state abbreviation in your SELECT line, so as to label the totals.

In [33]:
```
%%sql
SELECT state, COUNT(*)
FROM Customer
GROUP BY state;
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[33]:

state	COUNT(*)
AK	2
AL	1
CA	2
CT	1
FL	1
HI	1
IA	1
ID	1
IL	1
IN	1
LA	1
MI	1
MO	1
MS	1
MT	1
NC	2
NE	1
NY	1
OK	1
RI	1
TN	1
TX	1
UT	1
VA	1
VT	1
WA	1
WI	1

In [34]:
```
# Checking Your Results
```

4. Multiple-table queries

As you saw earlier, the review table was not easy to understand, as the book name was not in the query. You will perform a few multiple-table queries to explore the JOIN condition.

Step 4.1: Review existing Review and Book table

Re-run the queries from earlier:

In [35]:
```
%%sql
SELECT *
FROM Review;
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[35]:

review_id	rating	comment	book_id	customer_id
1	5	A timeless classic!	1	3
2	3	Interesting, but not my favorite.	3	17
3	4	Thought-provoking and impactful.	4	22
4	2	Did not connect with the characters.	5	7
5	5	An epic journey!	6	12
6	4	Magical and captivating.	7	25
7	3	A fun adventure.	8	6
8	5	Kept me guessing until the end!	9	15
9	2	Too predictable.	10	8
10	4	Beautiful and heartwarming.	11	19
11	5	Inspirational and thought-provoking.	12	2
12	4	Hilarious and absurd.	13	30
13	3	A childhood favorite.	14	11
14	5	A powerful allegory.	15	21
15	4	A classic for a reason.	16	5
16	2	Did not enjoy the writing style.	17	14
17	5	A masterpiece of literature.	18	28
18	3	Tragic and moving.	19	9
19	4	An epic tale.	20	16
20	5	A must-read for Python enthusiasts.	1	23
21	4	Great for learning the basics.	1	12
22	3	Could be more concise.	1	6
23	5	Excellent reference material.	1	20
24	4	A thought-provoking read.	3	15
25	3	Not as good as the first book.	6	24
26	5	The best in the series.	7	18
27	2	Disappointing ending.	9	5
28	4	A page-turner.	10	27
29	5	A classic whodunit.	9	13
30	3	Slow start, but picks up.	12	26

review_id	rating	comment	book_id	customer_id
31	4	Beautifully written.	11	3
32	5	Changed my perspective.	12	10
33	2	Confusing plot.	13	1
34	4	Loved the characters.	14	29
35	3	Important message.	15	22
36	5	A timeless story.	16	17
37	2	Too long.	17	4
38	4	Gripping and emotional.	18	20
39	3	Left me wanting more.	19	8
40	5	A classic adventure.	20	11

Given what you see here, it is impossible to see who left the review, or what book each review is for.

In [36]:
```
%%sql
SELECT *
FROM Book;
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[36]:

book_id	title	author
1	The Great Gatsby	F. Scott Fitzgerald
2	To Kill a Mockingbird	Harper Lee
3	Pride and Prejudice	Jane Austen
4	1984	George Orwell
5	The Catcher in the Rye	J.D. Salinger
6	The Lord of the Rings	J.R.R. Tolkien
7	Harry Potter and the Sorcerer's Stone	J.K. Rowling
8	The Hobbit	J.R.R. Tolkien
9	And Then There Were None	Agatha Christie
10	The Da Vinci Code	Dan Brown
11	The Little Prince	Antoine de Saint-Exupéry
12	The Alchemist	Paulo Coelho
13	The Hitchhiker's Guide to the Galaxy	Douglas Adams
14	The Chronicles of Narnia	C.S. Lewis
15	Animal Farm	George Orwell
16	Jane Eyre	Charlotte Brontë
17	Moby Dick	Herman Melville
18	War and Peace	Leo Tolstoy
19	Hamlet	William Shakespeare
20	The Odyssey	Homer

Given what you see here, you cannot tell how each book was reviewed.

Step 4.2: Join the Book and Review table

In order to combine data from two tables, they need to have a common field. Notice both the Book table and Review table contain a common field, book_id. This can be used to join the tables.

Join the two tables to see the Book information with each Review.

In [37]:
```
%%sql
SELECT *

FROM Book
```

```
JOIN Review ON Book.book_id = Review.book_id;
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[37]:

book_id	title	author	review_id	rating	comment	book_id_1	customer_id
1	The Great Gatsby	F. Scott Fitzgerald	1	5	A timeless classic!	1	3
3	Pride and Prejudice	Jane Austen	2	3	Interesting, but not my favorite.	3	17
4	1984	George Orwell	3	4	Thought-provoking and impactful.	4	22
5	The Catcher in the Rye	J.D. Salinger	4	2	Did not connect with the characters.	5	7
6	The Lord of the Rings	J.R.R. Tolkien	5	5	An epic journey!	6	12
7	Harry Potter and the Sorcerer's Stone	J.K. Rowling	6	4	Magical and captivating.	7	25
8	The Hobbit	J.R.R. Tolkien	7	3	A fun adventure.	8	6
9	And Then There Were None	Agatha Christie	8	5	Kept me guessing until the end!	9	15
10	The Da Vinci Code	Dan Brown	9	2	Too predictable.	10	8
11	The Little Prince	Antoine de Saint-Exupéry	10	4	Beautiful and heartwarming.	11	19
12	The Alchemist	Paulo Coelho	11	5	Inspirational and thought-provoking.	12	2
13	The Hitchhiker's Guide to the Galaxy	Douglas Adams	12	4	Hilarious and absurd.	13	30
14	The Chronicles of Narnia	C.S. Lewis	13	3	A childhood favorite.	14	11
15	Animal Farm	George Orwell	14	5	A powerful allegory.	15	21
16	Jane Eyre	Charlotte Brontë	15	4	A classic for a reason.	16	5

book_id	title	author	review_id	rating	comment	book_id_1	customer_id
17	Moby Dick	Herman Melville	16	2	Did not enjoy the writing style.	17	14
18	War and Peace	Leo Tolstoy	17	5	A masterpiece of literature.	18	28
19	Hamlet	William Shakespeare	18	3	Tragic and moving.	19	9
20	The Odyssey	Homer	19	4	An epic tale.	20	16
1	The Great Gatsby	F. Scott Fitzgerald	20	5	A must-read for Python enthusiasts.	1	23
1	The Great Gatsby	F. Scott Fitzgerald	21	4	Great for learning the basics.	1	12
1	The Great Gatsby	F. Scott Fitzgerald	22	3	Could be more concise.	1	6
1	The Great Gatsby	F. Scott Fitzgerald	23	5	Excellent reference material.	1	20
3	Pride and Prejudice	Jane Austen	24	4	A thought-provoking read.	3	15
6	The Lord of the Rings	J.R.R. Tolkien	25	3	Not as good as the first book.	6	24
7	Harry Potter and the Sorcerer's Stone	J.K. Rowling	26	5	The best in the series.	7	18
9	And Then There Were None	Agatha Christie	27	2	Disappointing ending.	9	5
10	The Da Vinci Code	Dan Brown	28	4	A page-turner.	10	27
9	And Then There Were None	Agatha Christie	29	5	A classic whodunit.	9	13
12	The Alchemist	Paulo Coelho	30	3	Slow start, but picks up.	12	26

book_id	title	author	review_id	rating	comment	book_id_1	customer_id
11	The Little Prince	Antoine de Saint-Exupéry	31	4	Beautifully written.	11	3
12	The Alchemist	Paulo Coelho	32	5	Changed my perspective.	12	10
13	The Hitchhiker's Guide to the Galaxy	Douglas Adams	33	2	Confusing plot.	13	1
14	The Chronicles of Narnia	C.S. Lewis	34	4	Loved the characters.	14	29
15	Animal Farm	George Orwell	35	3	Important message.	15	22
16	Jane Eyre	Charlotte Brontë	36	5	A timeless story.	16	17
17	Moby Dick	Herman Melville	37	2	Too long.	17	4
18	War and Peace	Leo Tolstoy	38	4	Gripping and emotional.	18	20
19	Hamlet	William Shakespeare	39	3	Left me wanting more.	19	8
20	The Odyssey	Homer	40	5	A classic adventure.	20	11

In [38]:
```
# Checking Your Results
```

Step 4.3: Join the Customer and Review table

You can also see which customers wrote which review. Notice the Customer table and the Review table both contain a common field, customer_id. This can be used to join the tables.

Join the two tables to see the Customer information with each Review.

In [39]:
```
%%sql
SELECT *
FROM Customer
JOIN Review ON Customer.customer_id = Review.customer_id;
```

```
 * sqlite:///mydatabase.db
Done.
```

Out[39]:

customer_id	name	email	address	city	state	zip	phone	r
3	Li Wei		789 Oak St	New City	NY	10956	555-456-7890	
17	Seamus Reilly		3637 Willow St	Providence	RI	02903	555-789-0123	
22	Tao Lin		4647 Oak St	Helena	MT	59601	555-876-5432	
7	Elena Petrova		1617 Birch St	Anchorage	AK	99501	555-678-9012	
12	Dimitris Papadopoulos		2627 Maple St	Jackson	MS	39201	555-765-4321	
25	Ingrid Svensson		5253 Maple St	Kansas City	MO	64106	555-098-7654	
6	Kwame Asante		1415 Maple St	Baton Rouge	LA	70802	555-234-5678	
15	Pablo Gonzalez		3233 Oak St	Nashville	TN	37203	555-456-7890	
8	Pierre Dubois		1819 Cedar St	Boise	ID	83702	555-098-7654	
19	Nguyen Van		4041 Birch St	Salt Lake City	UT	84101	555-678-9012	
2	Aisha Khan		456 Elm St	Springfield	IL	60654	555-987-6543	
30	Sarah Jones		876 Elm St	Beverly Hills	CA	90210	555-999-8888	
11	Fatima Ali		2425 Willow St	Honolulu	HI	96813	555-345-6789	
21	Maria Garcia		4445 Elm St	Hartford	CT	06106	555-543-2109	

customer_id	name	email	address	city	state	zip	phone	r
5	Kimiko Tanaka		1213 Willow St	Midland	TX	79701	555-789-0123	
14	Ola Nordmann		3031 Cedar St	Madison	WI	53703	555-987-6543	
28	Kofi Kingston		5859 Oak St	Montpelier	VT	05602	555-345-6789	
9	Ananya Patel		2021 Oak St	Charlotte	NC	28202	555-567-8901	
16	Nkechi Okonkwo		3435 Pine St	Olympia	WA	98501	555-321-0987	
23	Abdul Rahman		4849 Pine St	Indianapolis	IN	46204	555-210-9876	
12	Dimitris Papadopoulos		2627 Maple St	Jackson	MS	39201	555-765-4321	
6	Kwame Asante		1415 Maple St	Baton Rouge	LA	70802	555-234-5678	
20	Zahara Ben-Ami		4243 Cedar St	Tallahassee	FL	32301	555-098-7654	
15	Pablo Gonzalez		3233 Oak St	Nashville	TN	37203	555-456-7890	
24	Sofia Papadopoulou		5051 Willow St	Juneau	AK	99801	555-654-3210	
18	Astrid Johansen		3839 Maple St	Richmond	VA	23219	555-234-5678	
5	Kimiko Tanaka		1213 Willow St	Midland	TX	79701	555-789-0123	
27	Francesca Ferrari		5657 Cedar St	Montgomery	AL	36104	555-890-1234	

customer_id	name	email	address	city	state	zip	phone	r
13	Ayumi Sato		2829 Birch St	Lincoln	NE	68508	555-123-4567	
26	Jamal Malik		5455 Birch St	Lansing	MI	48911	555-567-8901	
3	Li Wei		789 Oak St	New City	NY	10956	555-456-7890	
10	Alessandro Rossi		2223 Pine St	Des Moines	IA	50309	555-890-1234	
1	John Smith		123 Main St	Springfield	NC	27263	555-123-4567	
29	Isabella Costa		6061 Pine St	Oklahoma City	OK	73102	555-765-4321	
22	Tao Lin		4647 Oak St	Helena	MT	59601	555-876-5432	
17	Seamus Reilly		3637 Willow St	Providence	RI	02903	555-789-0123	
4	Carlos Rodriguez		1011 Pine St	Sunnyvale	CA	94086	555-321-0987	
20	Zahara Ben-Ami		4243 Cedar St	Tallahassee	FL	32301	555-098-7654	
8	Pierre Dubois		1819 Cedar St	Boise	ID	83702	555-098-7654	
11	Fatima Ali		2425 Willow St	Honolulu	HI	96813	555-345-6789	

In [40]:
```
# Checking Your Results
```

Hints & Tips

- **Review** the previous reading on SQL Concepts. Examples are intentionally similar to the examples in that reading.

End of Lab

By completing this lab, you've gained experience in writing queries. Though the focus of this course is not SQL, having some experience in this area is extremely important in the field of data science.